

Lab 11: BACKUP / RESTORE / Schedules & Cross-Cluster DR Drill (75 min)

Learning Objectives

By the end of this lab you will be able to:

- Take full, incremental, and scheduled backups and read backup metadata without restoring
- Restore a database, a single table, and a point in time — into a new name, side by side
- Run a **cross-cluster DR drill**: back up from cluster A, restore into cluster B, verify
- Compute the RTO and RPO your backup cadence actually guarantees, and measure the real RTO
- Use `SHOW BACKUP` and revision history to answer "what did this table look like at 14:02?"
- Write a DR runbook with the specific commands your on-call will run at 3am

Prerequisites

- **Docker Desktop** (or Docker Engine) running — there is no `cockroach` binary to install
- ~4 GB free RAM (two small clusters run side by side in Part D)

Setup — Cluster A (primary)

```
scripts/crdb up
export A='postgres://root@localhost:26257?sslmode=disable'
```

The cluster runs in Docker (see [Lab 1](#)). From your machine it is `localhost:26257`; from inside another container it is `crdb1:26257`. `scripts/crdb run ...` executes inside node 1.

Backup storage in this lab. We use a local `node:local://` path so the lab runs offline. In production the destination is cloud storage: `s3://bucket/path?AUTH=implicit`, `gs://bucket/path?AUTH=implicit`, or `azure-blob://...`. The SQL is identical apart from the URI.

⚠ Licensing — read this before Part A

On a cluster with **no enterprise licence**, the backup features split like this (verified on v23.2). A licence is free for training — see option 3 below — but the lab is written so that every lesson lands without one:

Feature	Without a license
Full <code>BACKUP / RESTORE</code>	☐ works
<code>RESTORE ... WITH new_db_name / into_db</code>	☐ works
Full-cluster <code>BACKUP / RESTORE</code> (the DR drill)	☐ works
<code>CREATE SCHEDULE</code>	☐ works — full backups only
<code>WITH revision_history</code>	☐ enterprise
Incremental (<code>INTO LATEST IN ...</code>)	☐ enterprise
Point-in-time <code>RESTORE ... AS OF SYSTEM TIME</code>	☐ enterprise (needs revision history)

The error is explicit, e.g.: `use of BACKUP with revision_history requires an enterprise license`.

Three ways to run this lab:

1. **Free path (default).** Do every step; where a step is marked ☐ **Enterprise**, read it and run the free alternative given beside it. You still complete the DR drill end to end.
2. **A containerised demo cluster.**
`docker run --rm -it cockroachdb/cockroach:v23.2.5 demo --nodes 3 --no-example-database --empty` ships with a temporary licence, so every step works. Use it for Parts A–C; Part D needs two clusters, so start a second demo on other ports.
3. **A licence — free for training.** Cockroach Labs issues them at no cost for training and evaluation, so ask your instructor whether the class has one. Export it before starting the cluster and `scripts/crdb up` applies it for you: It ships with the repo (`.license.env`): `scripts/crdb up` prints `enterprise licence applied` when it starts, and nothing further is needed. Already have a cluster running? Apply it in place instead:

```
SET CLUSTER SETTING enterprise.license = 'crl-0-...';
```

(No `cluster.organization` — trial keys are bound to an empty organization name, and setting one invalidates them.)

Everything Parts D and E teach — the cross-cluster drill, the verification, the measured RTO, the runbook — works on the free path.

Create a dataset with something worth losing:

```
scripts/crdb sql <<'SQL'
CREATE DATABASE bank;
USE bank;

CREATE TABLE accounts (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name    STRING NOT NULL,
  balance DECIMAL(12,2) NOT NULL,
  region  STRING NOT NULL
);

CREATE TABLE transfers (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  from_id UUID NOT NULL REFERENCES accounts(id),
  to_id   UUID NOT NULL REFERENCES accounts(id),
  amount  DECIMAL(12,2) NOT NULL,
  ts      TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO accounts (name, balance, region)
SELECT 'user-' || g, 1000.00, (ARRAY['us-east','us-west','eu-west'])[1 + g % 3]
FROM generate_series(1, 5000) g;
SQL
```

Tasks

Part A: Full, Incremental & Revision-History Backups (15 min)

1. Full backup (free):

```
BACKUP DATABASE bank INTO 'nodelocal://1/backups/bank' AS OF SYSTEM TIME '-10s';
```

`AS OF SYSTEM TIME '-10s'` makes the backup read from a slightly stale snapshot, which avoids contending with live traffic. Use it on every production backup.

⚠ **If you run this within 10 seconds of creating the database**, it fails with `database "bank" does not exist, or invalid RESTORE time` — the snapshot is from before the database existed. Wait a few seconds, or drop the `AS OF SYSTEM TIME` clause for this first backup. Harmless in the lab; worth knowing when you script a backup right after a migration creates a table.

❑ **Enterprise:** add revision history so any timestamp inside the window is restorable:

```
BACKUP DATABASE bank INTO 'nodelocal://1/backups/bank'
  AS OF SYSTEM TIME '-10s'
  WITH revision_history;
```

Without a licence this returns `use of BACKUP with revision_history requires an enterprise license` — run the free form above and continue.

2. Inspect without restoring:

```
SHOW BACKUPS IN 'nodelocal://1/backups/bank';  
SHOW BACKUP FROM LATEST IN 'nodelocal://1/backups/bank';  
SHOW BACKUP FROM LATEST IN 'nodelocal://1/backups/bank' WITH privileges;
```

Note the `start_time` / `end_time` columns — that window is your RPO for this backup.

3. Change data, then take an incremental — ☐Enterprise. On the free path, take another *full* backup into the same collection instead; the rest of the lab works either way.

```
INSERT INTO transfers (from_id, to_id, amount)  
SELECT a.id, b.id, 10.00  
FROM (SELECT id FROM accounts LIMIT 500) a,  
      (SELECT id FROM accounts LIMIT 1) b;  
  
UPDATE accounts SET balance = balance - 10 WHERE region = 'us-east';  
  
-- Enterprise:  
BACKUP DATABASE bank INTO LATEST IN 'nodelocal://1/backups/bank' WITH revision_history;  
  
-- Free alternative – a second full backup in the same collection:  
BACKUP DATABASE bank INTO 'nodelocal://1/backups/bank' AS OF SYSTEM TIME '-10s';
```

4. Read the chain:

```
SHOW BACKUP FROM LATEST IN 'nodelocal://1/backups/bank';
```

One full plus one incremental. The incremental only stores what changed — check the `rows` and `size_bytes` columns to see how much smaller it is.

5. Record a timestamp you can restore back to:

```
SELECT cluster_logical_timestamp() AS marker;
```

Save this value. Then do something regrettable:

```
-- `transfers` has a foreign key onto `accounts`, so the dependent rows go first.  
-- (Try it without this line to see the FK error a real "oops" would hit.)  
DELETE FROM transfers  
WHERE from_id IN (SELECT id FROM accounts WHERE region = 'us-east')  
      OR to_id  IN (SELECT id FROM accounts WHERE region = 'us-east');  
  
DELETE FROM accounts WHERE region = 'us-east';  
SELECT count(*) FROM accounts;
```

The FK is doing its job here. It is also why a partial restore of one table is rarely enough: the tables that reference it have to come back consistently, which is the argument for database- or cluster-level restores over table-level ones.

Part B: Restore — Database, Table, and Point in Time (15 min)

Before you start Part B, note that any backup you took `AS OF SYSTEM TIME '-10s'` reflects the cluster as of ten seconds *before* you ran it. Take a fresh backup now and wait out that window, or the restore below will hand you back the rows you just deleted and you'll think the restore misbehaved. That gap is your RPO — Part C makes you put a number on it.

1. **Restore the whole database side by side** (never straight over the live one during a drill):

```
RESTORE DATABASE bank FROM LATEST IN 'nodelocal://1/backups/bank'
  WITH new_db_name = 'bank_restored';

SELECT count(*) FROM bank_restored.public.accounts;
```

2. **Restore a single table:**

```
CREATE DATABASE bank_table_only;
RESTORE TABLE bank.public.accounts
  FROM LATEST IN 'nodelocal://1/backups/bank'
  WITH into_db = 'bank_table_only';

SELECT count(*) FROM bank_table_only.public.accounts;
```

Restoring `transfers` alone would fail — it has a foreign key into `accounts`. Either restore both tables together or add `WITH skip_missing_foreign_keys`.

3. **Point-in-time restore using the marker from Part A** — ☐ **Enterprise** (it reads the revision history).

Without it you get: `invalid RESTORE timestamp: restoring to arbitrary time requires that BACKUP was created with revision_history`. On the free path, skip to step 4; the deleted rows are recoverable from the full backup you took *before* the delete, which is the free-tier version of the same lesson: your RPO is your backup interval.

```
RESTORE DATABASE bank
  FROM LATEST IN 'nodelocal://1/backups/bank'
  AS OF SYSTEM TIME '<marker value>'
  WITH new_db_name = 'bank_pit';

SELECT count(*) FROM bank_pit.public.accounts WHERE region = 'us-east';
```

The deleted rows are back. `revision_history` is what makes any timestamp within the backup window restorable, not just the backup instants.

4. **Time the restore — this is your measured RTO:**

```
SELECT job_id, job_type, status,
       finished - started AS duration,
       (SELECT sum(row_count) FROM [SHOW JOBS] WHERE job_id = j.job_id) AS rows
FROM [SHOW JOBS] j
WHERE job_type = 'RESTORE' ORDER BY created DESC LIMIT 5;
```

Dataset size	Restore duration	Implied restore rate
--------------	------------------	----------------------

Extrapolate: at that rate, how long does your 2 TB production database take? That number is your **actual** RTO, and it is usually much larger than the one in the runbook.

Part C: Scheduled Backups & RPO Math (10 min)

1. Create a schedule:

```
-- With a licence: incrementals every 5 minutes, a full every day
CREATE SCHEDULE bank_backups
FOR BACKUP DATABASE bank INTO 'nodelocal://1/backups/scheduled'
RECURRING '* /5 * * * *'
FULL BACKUP '@daily'
WITH SCHEDULE OPTIONS first_run = 'now';
```

On the free path this succeeds but prints:

Without an enterprise license, this schedule will only run full backups. Make that explicit rather than leaving a surprise in the notice:

```
CREATE SCHEDULE bank_backups_full
FOR BACKUP DATABASE bank INTO 'nodelocal://1/backups/scheduled'
RECURRING '@hourly'
FULL BACKUP ALWAYS
WITH SCHEDULE OPTIONS first_run = 'now';
```

This is a real capacity-planning point, not a licensing footnote. Full-only backups make every cycle cost a complete copy — so on the free tier your practical RPO is bounded by how often you can afford a full backup, not by how often you'd like one.

2. Inspect it:

```
SHOW SCHEDULES;
SELECT id, label, schedule_status, next_run, recurrence FROM [SHOW SCHEDULES]
WHERE label LIKE 'bank%';
```

Two rows appear: one for the full-backup schedule and one for the incremental.

3. Watch it fire, then check the jobs it created:

```
SELECT job_id, status, created, description
FROM [SHOW JOBS] WHERE job_type = 'BACKUP' ORDER BY created DESC LIMIT 5;
```

4. The RPO math. With incrementals every 5 minutes:

worst-case RPO = incremental interval + backup duration + detection time

If the incremental takes 40 s and it takes you 2 minutes to notice the outage, your true RPO is roughly $5 \text{ min} + 40 \text{ s} + 2 \text{ min} \approx 7.7 \text{ min}$ — not 5 minutes.

Cadence	Nominal RPO	Realistic RPO	Storage cost
Hourly incremental	1 h		low
15-minute incremental	15 m		medium
5-minute incremental	5 m		high
Continuous (changefeed to object store)	seconds		highest

5. Pause and drop the schedule so it stops firing for the rest of the lab:

```
PAUSE SCHEDULES SELECT id FROM [SHOW SCHEDULES] WHERE label LIKE 'bank%';
DROP SCHEDULES SELECT id FROM [SHOW SCHEDULES] WHERE label LIKE 'bank%';
```

Part D: Cross-Cluster DR Drill (25 min)

The drill that matters: cluster A is gone, bring the data up on cluster B.

1. **Start the standby cluster.** It is the same compose stack under a second project name, on different ports (SQL 26357, console 8180), so both clusters run side by side:

```
CRDB_COMPOSE=docker/labs-b.yml scripts/crdb up
```

```
scripts/crdb ps                                # cluster A
CRDB_COMPOSE=docker/labs-b.yml scripts/crdb ps  # cluster B
```

Both clusters mount the same backup volume at `/backups` (node 1 of each, via `--external-io-dir`). That is deliberate: it stands in for the cloud bucket both clusters would share in production, and it means the drill exercises **restore** rather than file copying.

2. **Take a full cluster backup on A** — cluster backups include users, roles, and settings, which a database backup does not:

```
scripts/crdb sql -e "
CREATE USER app_user;
GRANT CONNECT ON DATABASE bank TO app_user;"
```

```
# `AS OF SYSTEM TIME '-10s'` reads the cluster as it was ten seconds ago – which is
# before you created app_user. Wait the window out, or the restore comes back without
# the user you just added and step 5 looks like a restore bug.
sleep 12
scripts/crdb sql -e "BACKUP INTO 'nodelocal://1/dr/cluster' AS OF SYSTEM TIME '-10s';"
```

A historical backup is historical about *everything*, users and grants included. This is the same trap in production: back up "as of ten minutes ago" right after an emergency grant, and the grant is not in the backup. □With a licence, add `WITH revision_history`.

3. Confirm cluster B can see the backup — no copying required:

```
CRDB_COMPOSE=docker/labs-b.yml scripts/crdb sql -e \
"SHOW BACKUPS IN 'nodelocal://1/dr/cluster';"
```

4. Restore onto B and time it:

```
time CRDB_COMPOSE=docker/labs-b.yml scripts/crdb sql -e \
"RESTORE FROM LATEST IN 'nodelocal://1/dr/cluster';"
```

A full-cluster `RESTORE` must run into a cluster with no user data — which is exactly the DR situation. If B already has the `bank` database from an earlier attempt, reset it with `CRDB_COMPOSE=docker/labs-b.yml scripts/crdb reset`.

5. Verify the restore, don't assume it:

```
CRDB_COMPOSE=docker/labs-b.yml scripts/crdb sql -e "
SHOW DATABASES;
SELECT count(*) AS accounts FROM bank.public.accounts;
SELECT count(*) AS transfers FROM bank.public.transfers;
SELECT sum(balance) AS total_balance FROM bank.public.accounts;"
```

```
CRDB_COMPOSE=docker/labs-b.yml scripts/crdb sql -e "SHOW USERS;"
```

6. Compare against A — a DR drill without a comparison is theatre:

```
echo "A:"; scripts/crdb sql --format=tsv -e \
"SELECT count(*), sum(balance) FROM bank.public.accounts;"
echo "B:"; CRDB_COMPOSE=docker/labs-b.yml scripts/crdb sql --format=tsv -e \
"SELECT count(*), sum(balance) FROM bank.public.accounts;"
```

The row count **and** the business checksum must match. One without the other proves nothing.

7. Fill in the drill record:

Metric	Value
Backup size	
Backup duration	
Copy/transfer duration	
Restore duration	
Total measured RTO	
Row-count match A vs B	☐ / ☐
Checksum (sum of balances) match	☐ / ☐
Users and grants restored	☐ / ☐

Part E: The Runbook (10 min)

Write the runbook for this drill. It should be short enough to follow while adrenaline is high.

```
## DR Runbook – Restore `bank` to a Standby Cluster

**Trigger:** primary cluster unavailable for > 10 minutes, or data corruption confirmed.
**Owner:** on-call DBA. **Escalation:** platform lead after 30 minutes.

### 0. Decide (2 min)
- [ ] Is this a full-cluster loss (→ full restore) or a bad write (→ point-in-time restore)?
- [ ] Note the last known-good timestamp. `SELECT cluster_logical_timestamp();` if reachable.

### 1. Locate the backup (2 min)
    SHOW BACKUPS IN 's3://prod-backups/bank';
    SHOW BACKUP FROM LATEST IN 's3://prod-backups/bank';
- [ ] Confirm `end_time` is within RPO.

### 2. Restore (RTO: measured ____ min per TB)
    RESTORE FROM LATEST IN 's3://prod-backups/bank' AS OF SYSTEM TIME '<ts>';
- [ ] Watch: SELECT status, fraction_completed FROM [SHOW JOBS] WHERE job_type='RESTORE';

### 3. Verify BEFORE cutting traffic over
- [ ] Row counts match the last known-good figures
- [ ] Business checksum matches (e.g. SUM(balance))
- [ ] Users, roles, and grants present
- [ ] Application smoke test passes against the standby

### 4. Cut over
- [ ] Update connection string / DNS
- [ ] Confirm error rate returns to baseline
- [ ] Announce recovery, note actual RTO/RPO achieved

### 5. Post-incident
- [ ] Re-establish backup schedules on the new primary
- [ ] File the gap between measured and documented RTO
```

Swap runbooks with another pair. Can they follow yours without asking you a question?

Cleanup

```
scripts/crdb down  
CRDB_COMPOSE=docker/labs-b.yml scripts/crdb down
```

That also removes the shared `/backups` volume, so the next run of this lab starts clean.

Lab 11 Deliverables

□ **Backups:** full (and, with a licence, incremental + revision history), inspected via `SHOW BACKUP` □ **Restores:** whole database, single table, and point-in-time into new names □ **Schedule:** created, observed firing, and dropped; RPO computed honestly □ **DR drill:** cross-cluster restore completed and *verified* against the source □ **Measured RTO:** a real number, extrapolated to production data size □ **Runbook:** written, and readable by someone else at 3am

Challenge Exercises

1. **Locality-aware backups.** Take a backup with `BACKUP INTO ('s3://us-east/...?COCKROACH_LOCALITY=region%3Dus-east1', 's3://eu/...?COCKROACH_LOCALITY=default')`. What problem does this solve, and which regulation usually motivates it?
2. **Restore a table that has dropped since backup time.** Use `SHOW BACKUP` to find its name at that revision, then restore only it. What breaks if the schema has changed since?
3. **Backup a subset.** `BACKUP TABLE bank.accounts, bank.transfers INTO ...` and compare size and duration against the database backup. When is table-level worth the complexity?
4. **Measure the impact of `revision_history`.** Take the same backup with and without it. Compare size and duration. What did the extra bytes buy you?

Reference

Command	Purpose
<code>BACKUP INTO 'sink' AS OF SYSTEM TIME '-10s'</code>	Full backup from a stale snapshot
<code>BACKUP ... INTO LATEST IN 'sink'</code>	Incremental against the latest full
<code>WITH revision_history</code>	Enables arbitrary point-in-time restore in the window
<code>SHOW BACKUPS IN 'sink'</code>	List backup chains at a destination
<code>SHOW BACKUP FROM LATEST IN 'sink'</code>	Contents and timestamps without restoring
<code>RESTORE ... WITH new_db_name = 'x'</code>	Side-by-side restore, non-destructive
<code>RESTORE TABLE ... WITH into_db = 'x'</code>	Single-table restore
<code>RESTORE FROM LATEST IN 'sink'</code>	Full-cluster restore (users, roles, settings)
<code>CREATE SCHEDULE ... RECURRING ... FULL BACKUP ...</code>	Managed backup cadence
<code>SELECT cluster_logical_timestamp()</code>	Capture a restorable marker